



Paradigmas de Programación

Funciones de Orden Superior - Fold

Docente: Pennella Valeria

Función Foldl

❖ **foldl** es una función de **orden superior** que procesa una estructura de datos en un cierto orden y construye un valor de retorno. Se define de la siguiente forma:

```
foldl :: (a → b → a) → a → [b] → a
foldl _ valorNeutro [] = valorNeutro
foldl funcion valorNeutro (cab:cola) =
foldl funcion (funcion valorNeutro cab)
cola
```

Prelude> foldl (+) 0 [2, 3, 7, 5]

¿ Cómo se reduce ?

foldl (+) 0 [2, 3, 7, 5]

foldl (+) ((+) 0 2) [3, 7, 5]

foldl (+) 2 [3, 7, 5]

foldl (+) ((+) 2 3) [7, 5]

foldl (+) 5 [7, 5]

foldl (+) ((+) 5 7) [5]

foldl (+) 12 [5]

foldl (+) ((+) 12 5) []

foldl (+) 17 []

17

- Como podemos ver en este ejemplo, `foldl` va aplicando la función al valor neutro y a los elementos de la lista de izquierda a derecha, por eso es `foldl`.
- Hay otra función que funciona de manera similar, pero la gran diferencia esta en la función que recibe por argumento, la cual aplica primero un elemento de la lista y el segundo elemento la semilla. Es así que el **`foldr`** aplica la función a los elementos de la lista pero de derecha a izquierda.

Función Foldr

- Hay otra función que es foldr, en donde se reduce la lista de derecha a izquierda.

```
foldr :: (b → a → a) → a → [b] → a
foldr _ valorNeutro [] = valorNeutro
foldr funcion valorNeutro (cab:cola) =
  funcion cab (foldr funcion valorNeutro
cola)
```

```
>foldl (-) 0 [1, 2, 5]

foldl (-) ((-) 0 1) [2, 5]

foldl (-) -1 [2, 5]

foldl (-) ((-) (-1) 2) [5]

foldl (-) -3 [5]

foldl (-) ((-) (-3) 5) []

foldl (-) -8 []

-8
```

```
>foldr (-) 0 [1, 2, 5]

(-) 1 (foldr (-) 0 [2, 5])

(-) 1 ((-) 2 (foldr (-) 0 [5]))

(-) 1 ((-) 2 ((-) 5 (foldr (-)
0 [])) )

(-) 1 ((-) 2 ((-) 5 0))

(-) 1 ((-) 2 5)

(-) 1 -3

4
```

- Claramente llegamos a resultados diferentes:

```
>foldl1 (-) 0 [1,2,5]  
-8
```

```
>foldr (-) 0 [1,2,5]  
4
```

Foldl1/Foldr1

```
foldl1 :: (a -> a -> a) -> [a] -> a  
foldl1 f (x:xs) = foldl f x xs
```

Como vemos invoca a foldl y le pasa como valor inicial la cabeza de la lista. Es importante tener en cuenta que no podemos aplicar foldl1 a una lista vacía.

Algunos ejemplos:

Si queremos conocer la sumatoria de la lista usando foldl1 sería:

sumatoria lista = foldl1 (+) lista

Para conocer la productoria de una lista sería:

productoria lista = foldl1 (*) lista

Para buscar el máximo de la lista con foldl1, quedaría:

maximo lista = foldl1 max lista

Además tenemos la función `foldr1` que es una variante de `foldr`. Su definición es:

```
foldr1 :: (a -> a -> a) -> [a] -> a
```

```
foldr1 f [x]          = x
```

```
foldr1 f (x:xs)       = f x (foldr1 f xs)
```

Y tampoco podemos aplicar `foldr1` a una lista vacía.

Hay casos donde es indistinto usar `foldl1` o `foldr1` porque llegamos a los mismos resultados, pero hay otros que no. Veamos algunos ejemplos.

Casos donde con los mismos valores obtenemos resultados diferentes:

```
> foldl1 (-) [2, 4, 6]  
-8
```

```
> foldr1 (-) [2, 4, 6]  
4
```

```
> foldl1 div [8,4,2]  
1
```

```
> foldr1 div [8,4,2]  
4
```

Resumen

- Estudiamos las siguientes funciones de orden superior:
 - ❖ Foldl
 - ❖ Foldr
 - ❖ Foldl1
 - ❖ Foldr1